

UNIT 1: INTRODUCTION

1.1 Basic Technology and Elementary Data Organization

Basic Technology

Data Structure is a fundamental concept in computer science used to organize, store, and manage data efficiently. It helps in performing operations such as insertion, deletion, searching, and sorting in an efficient manner.

A computer stores data in **memory** in the form of bits and bytes. To process large amounts of data efficiently, data must be organized in a proper structure.

Elementary Data Organization

Elementary data organization refers to the basic ways in which data is represented and organized in a computer.

The basic units of data organization are:

1. **Bit** – The smallest unit of data. It can have a value of 0 or 1.
2. **Byte** – A group of 8 bits.
3. **Character** – A single alphabet, digit, or special symbol.
4. **Field** – A collection of related characters representing one attribute.
5. **Record** – A collection of related fields.
6. **File** – A collection of related records.
7. **Database** – A collection of logically related data stored and managed together.

Example:

A student's information can be organized as:

Database → File → Record → Fields

Student Record:

- Roll No: 101
- Name: Rahul
- Course: BCS
- Marks: 85

1.2 Data Structure

A **data structure** is a particular way of organizing and storing data in a computer so that it can be accessed and processed efficiently.

In simple words, a data structure provides a systematic method for storing data and performing operations on it.

Examples of Data Structures

- Array
- Linked List
- Stack
- Queue
- Tree
- Graph

Classification of Data Structures

Data structures are mainly classified into two types:

1. Primitive Data Structures

These are the basic data types provided by a programming language.

Examples:

- Integer
- Float
- Character
- Boolean
- Pointer

2. Non-Primitive Data Structures

These are created using primitive data types.

They are further divided into:

A. Linear Data Structures

Data elements are arranged sequentially.

Examples:

- Array
- Linked List
- Stack
- Queue

B. Non-Linear Data Structures

Data elements are not arranged sequentially.

Examples:

- Tree

- Graph

Importance of Data Structures

Data structures are important because they:

- Organize data efficiently.
 - Make data access faster.
 - Reduce memory usage.
 - Improve program performance.
 - Make complex programs easier to design.
 - Help in efficient searching and sorting.
-

1.3 Data Structure Operations

Data structure operations are the basic operations performed on data stored in a data structure.

The major operations are:

1. Traversing

Traversing means visiting and processing each element of a data structure exactly once.

Example: Displaying all elements of an array.

2. Insertion

Insertion means adding a new element to a data structure.

Example: Adding a new element to an array.

3. Deletion

Deletion means removing an existing element from a data structure.

Example: Removing an element from a list.

4. Searching

Searching means finding a particular element in a data structure.

Common searching methods:

- Linear Search
- Binary Search

5. Sorting

Sorting means arranging data elements in a specific order.

Examples:

- Ascending order

- Descending order

Common sorting methods include:

- Bubble Sort
- Selection Sort
- Insertion Sort

6. Merging

Merging means combining two or more similar data structures into a single data structure.

7. Updating

Updating means modifying the value of an existing element.

1.4 Complexity and Time-Space Tradeoff

Complexity

The complexity of an algorithm describes the amount of resources required by an algorithm as the input size increases.

The two main types of complexity are:

1. Time Complexity

Time complexity measures the amount of time required by an algorithm to execute.

It is generally represented using **Big O notation (O)**.

Common time complexities are:

Complexity Name	Example	
$O(1)$	Constant	Accessing an array element
$O(\log n)$	Logarithmic	Binary Search
$O(n)$	Linear	Linear Search
$O(n \log n)$	Linearithmic	Efficient sorting algorithms
$O(n^2)$	Quadratic	Bubble Sort

2. Space Complexity

Space complexity measures the amount of memory required by an algorithm during execution.

It includes:

- Memory required for input data.
- Memory required for variables.

- Extra memory required during execution.

Time-Space Tradeoff

Time-space tradeoff is a technique where we sacrifice one resource to improve another.

In simple words:

More memory can be used to reduce execution time, or less memory can be used at the cost of more execution time.

Example

Suppose a program stores previously calculated results in memory. This requires extra memory but avoids recalculating the same results, making the program faster.

Therefore:

More Space → Less Time

or

Less Space → More Time

The choice depends on the requirements and available resources.

1.5 Linear Array

An **array** is a linear data structure that stores a collection of elements of the same data type in contiguous memory locations.

A **linear array** is a one-dimensional array in which elements are arranged sequentially.

Example

Consider an array:

A = [10, 20, 30, 40, 50]

Here:

- A[0] = 10
- A[1] = 20
- A[2] = 30
- A[3] = 40
- A[4] = 50

The position of an element is called its **index** or **subscript**.

Characteristics of Linear Array

1. Elements are stored in a linear sequence.
2. All elements are generally of the same data type.

3. Elements are stored in contiguous memory locations.
4. Each element can be accessed using an index.
5. Array indexing usually starts from 0 in languages such as C, C++, Java, and Python.

Advantages

- Easy to access elements.
- Fast random access.
- Simple to implement.
- Efficient memory utilization.
- Suitable for storing large amounts of similar data.

Disadvantages

- Fixed size in many programming languages.
- Insertion and deletion can be time-consuming.
- Requires contiguous memory locations.

1.6 Representation of Linear Array in Memory

A linear array is stored in **contiguous memory locations**. This means that array elements are stored one after another in memory.

Consider:

A = [10, 20, 30, 40]

Suppose the starting address of array A is **1000** and each element requires **4 bytes**.

The memory representation will be:

Element Index Memory Address

10	0	1000
20	1	1004
30	2	1008
40	3	1012

Address Calculation Formula

For a one-dimensional array:

$LOC(A[K]) = \text{Base Address} + (K \times \text{Size of Each Element})$

Where:

- **$LOC(A[K])$** = Address of element at index K

- **Base Address** = Address of the first element
- **K** = Index of the required element
- **Size** = Memory size of each element in bytes

Example

Given:

- Base Address = 1000
- Element size = 4 bytes
- Required element = A[3]

Then:

$$\text{LOC(A[3])} = 1000 + (3 \times 4)$$

$$\text{LOC(A[3])} = 1012$$

Therefore, the address of A[3] is **1012**.

1.7 Traversing Linear Array

Array traversal is the process of visiting each element of an array exactly once, usually to display or process the elements.

Consider:

A = [10, 20, 30, 40, 50]

During traversal, elements are accessed one by one:

10 → 20 → 30 → 40 → 50

Algorithm for Traversing a Linear Array

Step 1: Start.

Step 2: Set index $i = 0$.

Step 3: Check whether $i < n$, where n is the number of elements.

Step 4: Access or process $A[i]$.

Step 5: Increase i by 1.

Step 6: Repeat Steps 3 to 5 until all elements are processed.

Step 7: Stop.

Example

For the array:

A = [5, 10, 15, 20]

Traversal output:

5 10 15 20

Time Complexity

The time complexity of traversing an array containing **n** elements is:

$O(n)$

Because every element is visited once.

1.8 Searching Methods

Searching is the process of finding a particular element, called the **key element**, in a data structure.

The two common searching methods are:

1. Linear Search
 2. Binary Search
-

A. Linear Search

Linear Search is a simple searching technique in which each element is checked one by one until the required element is found or the entire array is searched.

It can be used on both **sorted and unsorted arrays**.

Example

Consider:

A = [10, 25, 30, 45, 50]

Suppose we want to search for **45**.

The search process is:

10 → 25 → 30 → 45

The element **45** is found at index **3**.

Algorithm

1. Start from the first element.
2. Compare the current element with the search key.
3. If both are equal, return the position of the element.
4. If they are not equal, move to the next element.
5. Continue until the element is found or the array ends.
6. If the array ends without finding the element, report that the element is not present.

Advantages

- Simple and easy to implement.
- Works with sorted and unsorted data.
- Does not require additional memory.

Disadvantages

- Slow for large datasets.
- May need to check every element.

Complexity

- **Best Case:** $O(1)$
 - **Average Case:** $O(n)$
 - **Worst Case:** $O(n)$
-

B. Binary Search

Binary Search is an efficient searching technique used to find an element in a **sorted array**.

It repeatedly divides the search range into two halves.

Example

Consider the sorted array:

A = [10, 20, 30, 40, 50, 60, 70]

Suppose we want to search for **60**.

Step 1

Check the middle element:

40

Since **60 > 40**, search in the right half.

Step 2

Right half:

[50, 60, 70]

Middle element:

60

The element is found.

Algorithm

1. Ensure that the array is sorted.

2. Set low to the first index.
3. Set high to the last index.
4. Calculate the middle position:

$$\text{mid} = (\text{low} + \text{high}) / 2$$

5. Compare the middle element with the search key.
6. If the middle element equals the key, the element is found.
7. If the key is smaller, search the left half.
8. If the key is larger, search the right half.
9. Repeat the process until the element is found or the search range becomes empty.

Advantages

- Faster than linear search for large sorted datasets.
- Reduces the search area by half in every step.
- Efficient for large amounts of data.

Disadvantages

- Array must be sorted.
- More complex than linear search.
- Not suitable when data changes frequently and sorting must be repeated.

Complexity

- **Best Case:** $O(1)$
- **Average Case:** $O(\log n)$
- **Worst Case:** $O(\log n)$

Difference Between Linear Search and Binary Search

Feature	Linear Search	Binary Search
Data Requirement	Sorted or unsorted	Must be sorted
Searching Method	Checks elements one by one	Divides data into two halves
Best Case	$O(1)$	$O(1)$
Average Case	$O(n)$	$O(\log n)$
Worst Case	$O(n)$	$O(\log n)$
Speed	Slower for large data	Faster for large sorted data

Feature	Linear Search	Binary Search
Simplicity	Very simple	More complex
Suitable For	Small or unsorted data	Large sorted data

Conclusion

Data structures provide efficient methods for organizing and processing data. A **linear array** is one of the simplest linear data structures. It supports operations such as traversal and searching. **Linear Search** is simple and works on unsorted data, while **Binary Search** is much faster but requires the data to be sorted. Understanding complexity and the time-space tradeoff helps in selecting the most efficient algorithm for a particular problem.